Innlevering nr. 4, ING3504 Signalbehandling

Kap. 2

Oppgave nr. 1 Konvolusjon er distributiv

Ved å beregne for hånd på tre korte sekvenser skal du vise at konvolusjon er distributiv, dvs. at vanlige parentesregler gjelder.
Gitt tre sekvenser: $x_1[n]$ = {4, -1, 5 }, $x_2[n]$ = {6, -1, -1 } og x_3[n] = {-4, 3, 2 }
Beregn: $x_1[n]$ * ($x_2[n]$ + $x_3[n]$) og ($x_1[n]$ * $x_2[n]$ + $x_1[n]$ * $x_3[n]$) og vis at de er like.
I denne oppgaven betyr * konvolusjon (folding), (og ikke multiplikasjon).
Kontroller beregningene med Octave/Matlab (se koding nedenfor).

For å vise mer generell gyldighet, test ut litt lengre sekvenser i Octave/Matlab.
Gitt tre sekvenser: $x_1[n]$ = {4, -1, 0, 5 }, $x_2[n]$ = {6, -1, 7, -1, 1 } og x_3[n] = {-4, 3, 2, 1 }
Visse operasjoner krever at sekvensene er like lange – legg eventuelt til nuller i begynnelsen eller slutten der det er nødvendig.
Beregn: $x_1[n]$ * ($x_2[n]$ + $x_3[n]$) og ($x_1[n]$ * $x_2[n]$ + $x_1[n]$ * $x_3[n]$) og vis at de er like.
Tenk etter hvordan du enklest/best kan vise at sekvensene er like.

Koding for første delen:

```
x1=[4,-1,5];
x2=[6,-1,-1];
x3=[-4,3,2];
y1=conv(x1,(x2+x3))
y2=conv(x1,x2)+conv(x1,x3)      % Skriver ut for enkelt å se at y2 = y1
```

Oppgave nr. 2 Punktprøving/Sampling

Punktprøv (sample) det analoge signalet $x(t) = 2\cos(2\pi 100 t)$ med punktprøvingsfrekvensen (samplingsfrekvensen) f_s = 800 Hz. Finn et algebraisk uttrykk (bokstavuttrykk) for $x[n]$ og beregn sekvensen for én periode av signalet. Hva er den digitale periodelengden (antall sampelpunkter på en periode)? Beregninger gjøres "for hånd". Men beregn/plot gjerne også i Octave.

Kode for beregning/plotting:

```
f = 100;   N = 15;    % Frekvens.  N=antall sampelpunkter å beregne
fs = 800;  Ts = 1/fs; % Sampelfrekvens.  Ts=avstand mellom sampelpunkter
t=[0:Ts:(N-1)*Ts];    % tidsvektor
% t=[0:(N-1)]/fs;       % Alternativ beregning av tidsvektor
x = 2*cos(2*pi*f*t)     % Sampelverdier

figure; hold on;
plot(t*1000,x);  stem(t*1000,x);  hold off;  % Multipliser med 1000 for å få tid i ms.
xlabel('Time [ms]');  grid;      % Plotter også heldratt kurve mellom sampel-punktene.
```

Oppgave nr. 3 FIR-filter

Et digitalt filter har impulsrespons: $h[n]$={1, 3, 2}

a) Er dette et FIR-filter eller er det et IIR-filter? Begrunn svaret.
b) Tegn filterstrukturen.
c) Bestem differensligningen.

d) Bestem utsignalet om innsignalet er: $x[n]=\{2, 3, -1\}$

e) Bestem overføringsfunksjonen (systemfunksjonen) $H(z)$ med tilhørende poler og nullpunkt. Tegn pol-nullpunkts-diagram.

Oppgave nr. 4 IIR-filter

Et andre ordens digitalt filter har nullpunkter $z = -1$ og $z = -2$, og to poler (en dobbelt-pol) i $z = 0{,}8$.

a) Bestem overføringsfunksjonen (systemfunksjonen) $H(z)$.
b) Bestem differensligningen og tegn filterstrukturen.
c) Bestem utsignalet for $n \in [0, 6]$ om innsignalet er: $x[n]=\{2, 3, -1\}$. Bruk tabell.
d) Er filteret stabilt? Begrunn svaret.
e) Bestem impulsresponsen.
f) Test ulike koeffisientverdier. Kan man si noe om stabilitet for filteret direkte fra koeffisientverdiene?
Kontroller resultatet med vedlagte Octave-kode.

Octave-kode for noen av beregningene:

```
pkg load signal; % signal-pakken må evt lades
close all; clear all; clc; format long;

b=[1  3  2];       % b-vektor (tellerpolynom), (sml FIR-filter i oppg. 3)
a=[1 -1.6  0.64];  % a-vektor (nevnerpolynom)

nullpkt= roots(b),   abs_nullpkt = abs(nullpkt)
poler  = roots(a),   abs_poler = abs(poler)
zplane(b,a)        % Tegn evt Pol-nullpunkts-diagrammet

h=impz(b,a, 45);    % Impulsrespons (de 45 første verdiene)
% figure; stem(h);  % Plot evt bare implulsresponsen

x=[2 3 -1];     % Innsignal
y = conv(x,h);  % Utsignal gitt av konvolusjon
y= y(1:length(h));   % Begrens lengden til det samme lengde som h[n], for plotting
% Ly = length(y)     % Kontroller evt lengden på y-vektor

figure; hold on;
stem(h,'b');  stem(y,'r'); hold off;
grid;  yticks([-10:5:55]);
legend('h[n] - Impulsrespons', 'y[n] - Utsignal');


y(3:length(y))  % Tar bort de to første nullene og skriver ut resultatet
```

Oppgave nr. 5 Middelverdi, Rekursiv og ikke-rekursiv løsning

a) Vi ønsker å bruke et digitalt filter på datasekvensen: $x[n]=\{10,14, 2, 6, 8, 20,1,17,14,7,15,16\}$ ved å ta middelverdien av 3 verdier etter hverandre i sekvensen. Fyll inn nuller i starten av sekvensen som i kompendiet (kapittel 2.1.4). Lag et Matlab/Octave-program som realiserer filteret, først med ikke-rekursiv løsning og så med rekursiv løsning (se kompendiet, kapittel 2.1.4).

b) Med uendelig stor presisjon gir disse to løsningene identiske svar. Men om prosessoren har begrenset antall binære bit (ordlengde) gir de ulike resultat. Her skal vi dividere med 3 noe som er umulig å gjøre eksakt med binær aritmetikk. Med tre eller fire binære desimaler tilgjengelig kan vi representere multiplikasjon med: $2^{-2} + 2^{-3} = 0{,}375$ eller $2^{-2} + 2^{-4} = 0{,}3125$ (istedenfor 0,33333…). Fem binære desimaler: $2^{-2} + 2^{-4} + 2^{-5} = 0{,}34375$ Seks binære desimaler: $2^{-2} + 2^{-4} + 2^{-6} = 0{,}328125$

Prøv disse verdiene og se hvor mye det påvirker resultatet for de to realiseringene (rekursiv og ikke-rekursiv struktur). Blir det noen forskjell om vi multipliserer hver av inngangsverdiene først og summerer etterpå, sammenlignet med om vi summerer først og utfører bare én multiplikasjon? Effekten av begrenset ordlengde i prosessorene er viktig, men en videre analyse ligger utenfor vårt emne.

Koding for divisjon med 3 (med Octavs/Matlabs innebygde presisjon som er meget god).

```
xx=[10,14,2,6,8,20,1,17,14,7,15,16];
x = [zeros(1,3), xx, zeros(1,2)];  % Legger til '0-ere' i starten og slutten
% Må ha 3 '0-ere' i starten for den rekursive løsningen.
% '0-ere' kan også legges inn manuelt:
% x=[0, 0, 0, xx, 0, 0];

% a) IKKE-rekursiv løsning (utsignal y)
y = zeros(size(x));
for n=3:length(x) % kunne ha startet på n=4, første element ut som ikke er null
   y(n)=(x(n)+x(n-1)+x(n-2))/3;
end;

% b) Rekursiv løsning (utsignal yr)
yr = zeros(size(x));  % Samme lengde som y (for IKKE-rekursiv løsning)
for n=4:length(x)    % Må begynne for n=4 siden vi har x(n-3)
   yr(n)=yr(n-1) + (x(n)-x(n-3))/3;
end;

% Skriv ut de to resultatene og differensen for å se at de er like
y(1:8), yr(1:8)   % Skriver ut de 8 første elementene
Differens = y-yr  % Noen veldig små tall viser numerisk unøyaktighet

% ------------------------------------------------------------
% Alternativt, plot resultatene ('stem') og se at de er like.
% Behøver IKKE å spesifisere x-aksen
figure;
stem(y,'b.'); title('Ikke rekursiv løsning'); grid;
xlabel('n'); ylabel('Utsignal y(n)');
figure;
stem(yr,'r.'); title('Rekursiv løsning'); grid;
xlabel('n'); ylabel('Utsignal y(n)');

% Enklere å se eventuelle forskjeller om man plotter i samme diagram
figure; hold on;
stem(y,'b.'); stem(yr,'r.'); hold off;  grid;
legend('IKKE rekursiv', 'Rekursiv', 'Location','northwest');
xlabel('n'); ylabel('Utsignal');
```

Oppgave 6 Filtrering med FIR-filter

Et FIR-filter har koeffisientene
$b_0 = b_{10} = 0.02$
$b_1 = b_9 = 0.04$
$b_2 = b_8 = 0.08$
$b_3 = b_7 = 0.12$
$b_4 = b_6 = 0.15$
$b_5 = 0.16$

I dette tilfellet er filteret symmetrisk og kausalt.

a) Lag en linje-vektor b som inneholder disse elleve filterkoeffisientene:
b = [0.02 0.04 0.08 0.12 0.15 0.16 0.15 0.12 0.08 0.04 0.02];

b) Hvilken orden er dette filteret?
Den høyeste koeffisienten b_{10} hører til $x[n\text{-}10]$, som er inngangssignalet forsinket med 10 intervall. Filterorden er 10.

Filteret skal brukes til å filtrere et signal (en sekvens) med lengde 64.
Tidsreferansen skal være en heltalls linje-vektor {0, 1, 2, . . . ,63}, n = [0:63];
Lag en diskret sinussekvens med frekvensen $\hat{\omega} = 0.0625\pi$, x0625 = sin(0.0625*pi*n);
Lag et stolpediagram av signalet x0625, stem(n, x0625, 'b');
Sinussignalet skal filtreres med FIR-filteret, og utgangssignalet heter y0625.
Bruk funksjonen filter slik: y0625 = filter(b, 1, x0625);

(Merknad 1: Parameter 2 skal være 1 for et FIR-filter.)
(Merknad 2: *conv*-funksjonen kan brukes, men utgangssekvensen blir da lenger enn inngangssekvensen og den må kortes av for å plottes sammen med innsignalet.)

Skisser det filtrerte signalet i samme stolpediagram som inngangssignalet.
hold on; stem(n, y0625, 'r'); hold off;
For å få en finere inndeling langs vertikalaksen, bruk:
set(gca, 'ytick', -1:0.05:1); eller yticks([-1:0.05:1]);

c) Kopier stolpediagrammet (eller lagre grafikkfil/skjermkopi) og lim inn i svaret.
Hvor stor er amplituden til utsignalet y0625 sammenlignet med innsignalet x0625?

Octave-kode for første del:

```
clear all; clc; format compact;
pkg load signal;  % Brukes i Octave
n = [0:63];
b = [0.02 0.04 0.08 0.12 0.15 0.16 0.15 0.12 0.08 0.04 0.02];  % FIR

x0625 = sin(0.0625*pi*n);  % Diskret frekvens  pi/16
y0625 = filter(b,1,x0625);

figure; hold on; grid;
stem(n, x0625, 'b');   stem(n, y0625, 'r');    yticks([-1:0.1:1]);
% set(gca, 'ytick', -1:0.05:1); % Alternativ
% Merk 'ytick' og 'yticks'
hold off;
```